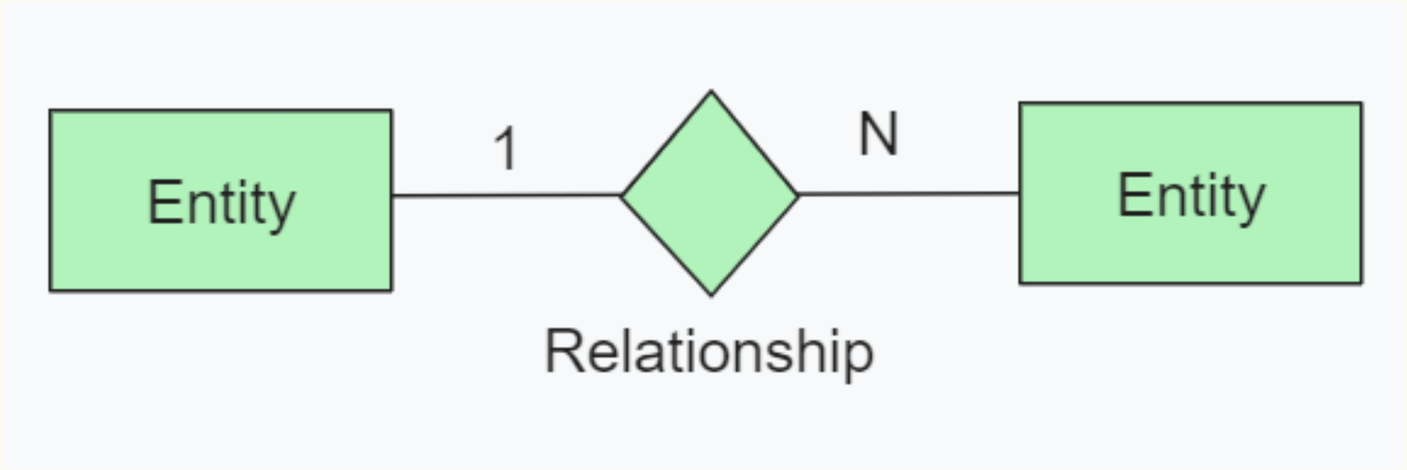
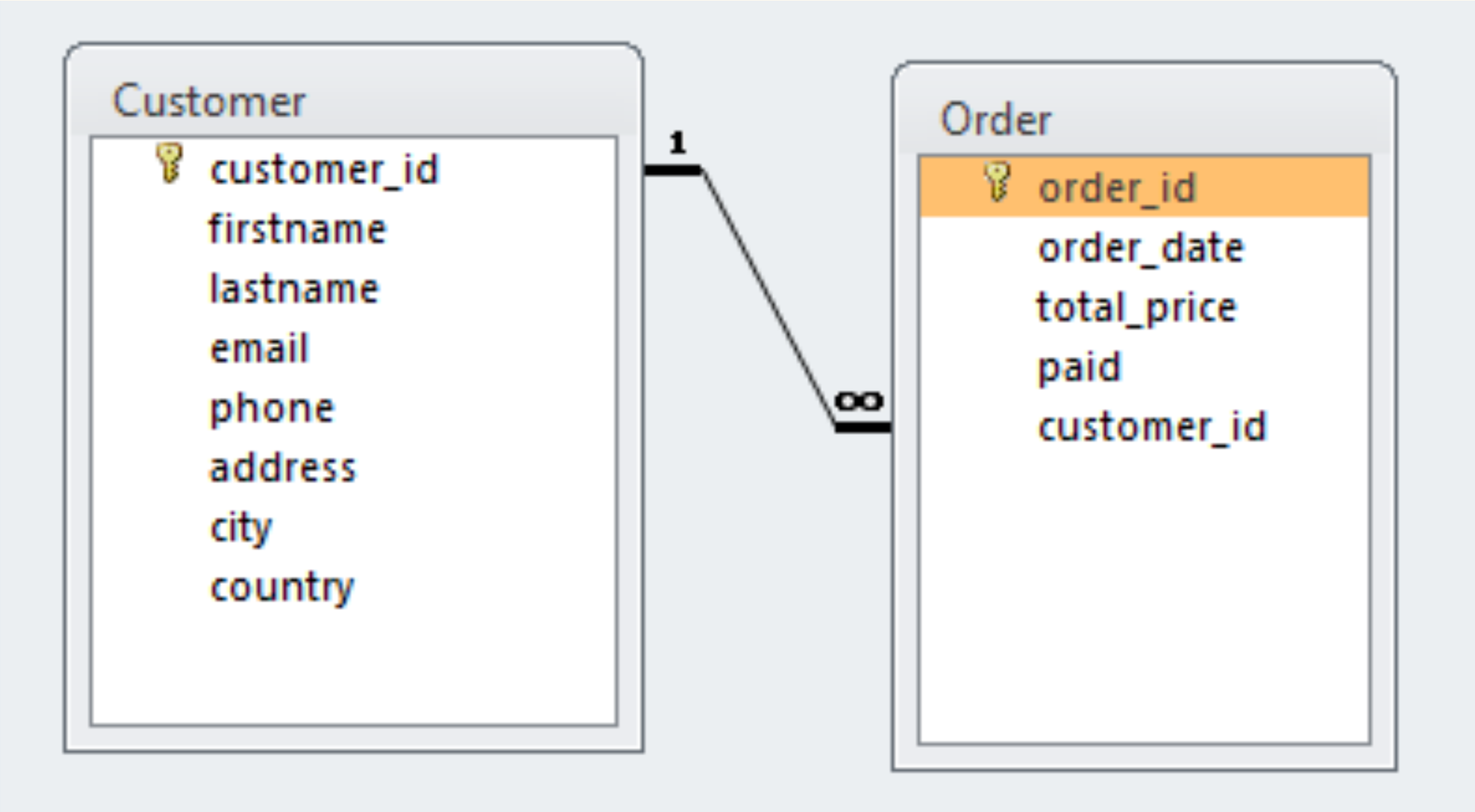
Blog App

I. Lesson

Today, we focus on One-to-Many (1->n)


Entity
1
N
Entity
Relationship


Customer
1
has
n
Orders


Customer
customer_id
firstname
lastname
email
phone
address
city
country
1
∞
Order
order_id
order_date
total_price
paid
customer_id

```sql
CREATE TABLE ParentTable (
    ParentID INT PRIMARY KEY,
    ParentAttribute DataType
);

CREATE TABLE ChildTable (
    ChildID INT PRIMARY KEY,
    ChildAttribute DataType,
    ParentID INT,
    FOREIGN KEY (ParentID) REFERENCES ParentTable(ParentID)
);
```

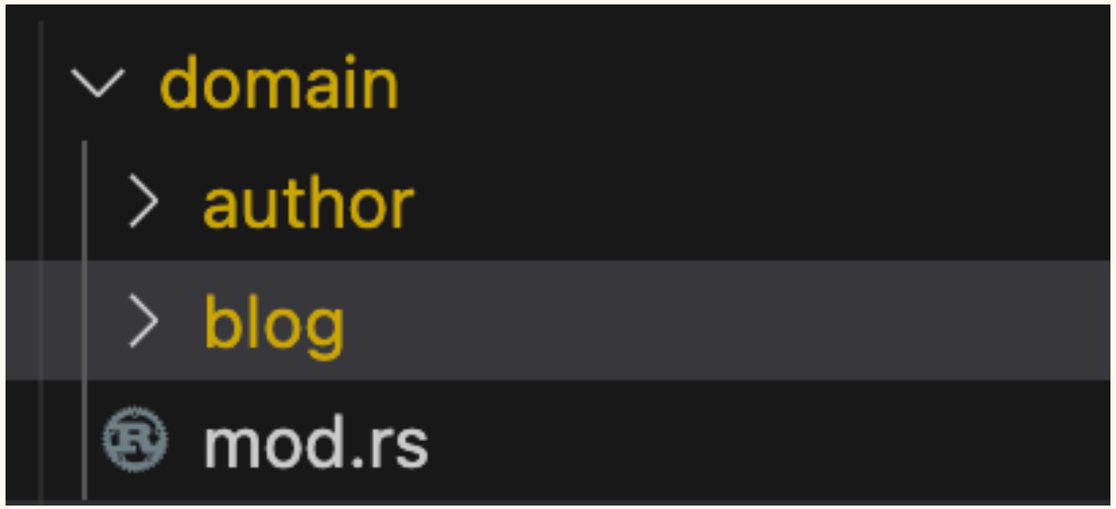
II. Project

Blog example

1 author -> n blogs

```sql
CREATE TABLE IF NOT EXISTS authors (
    id          SERIAL          PRIMARY KEY,
    name        VARCHAR(255)    NOT NULL,
    email       VARCHAR(255)    NOT NULL UNIQUE,
    created_at  TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);
```

```sql
CREATE TABLE IF NOT EXISTS blogs (
    id          SERIAL          PRIMARY KEY,
    title       VARCHAR(255)    NOT NULL,
    content     TEXT            NOT NULL,
    author_id   INTEGER         NOT NULL REFERENCES authors(id) ON DELETE CASCADE,
    created_at  TIMESTAMPTZ     NOT NULL DEFAULT NOW(),
    updated_at  TIMESTAMPTZ     NOT NULL DEFAULT NOW()
);
```


domain
author
blog
mod.rs

app.rs

```
let app = Router::new()
    .merge(authors_routes())
    .merge(blogs_routes())
    .with_state(app_state);
```

domain/blog/dto.rs

```rust
#[derive(Debug, Deserialize, Serialize)]
pub struct CreateBlogDto {
    pub title: String,
    pub content: String,
    pub author_id: i32,
}
```

A few reminders on Error handling

domain/blog/error.rs

```rust
#[derive(Debug, thiserror::Error)]
pub enum BlogError {
    #[error("Blog with id {0} not found")]
    NotFound(i32), // <- This creates a constructor function
    #[error("Database error: {0}")]
    Database(#[from] sqlx::Error),
}
```

domain/blog/error.rs

```rust
impl IntoResponse for BlogError {
    fn into_response(self) -> Response {
        let (status_code, error_message) = match self {
            BlogError::NotFound(_) => (StatusCode::NOT_FOUND, self.to_string()),
            BlogError::Database(ref e) => {
                tracing::error!("Database error: {}", e);
                (StatusCode::INTERNAL_SERVER_ERROR, "Internal server error".to_string())
            }
        };

        (status_code, Json(BlogApiError { error: error_message })).into_response()
    }
}
```

Axum

```rust
pub trait IntoResponse {
    /// Create a response.
    #[must_use]
    fn into_response(self) -> Response;
}
```

domain/blog/blog.rs

```rust
pub async fn get_by_id(Path(id): Path<i32>, app_state: State<AppState>) -> Result<Json<Blog>,
BlogError> {
    let blog = sqlx::query_as!(
        Blog,
        r#"
            ...
        "#,
        id
    )
    .fetch_one(&app_state.pool)
    .await
    .map_err(|e| match e {
        sqlx::Error::RowNotFound => BlogError::NotFound(id),
        _ => BlogError::Database(e),
    })?;

    Ok(Json(blog))
}
```

Sometimes, you won’t need map_err

Fetch all

```rust
pub async fn get_all(app_state: State<AppState>) -> Result<Json<Vec<Blog>>,
BlogError> {
    let blogs = sqlx::query_as!(
        Blog,
        r#"
            SELECT *
            FROM blogs
            ORDER BY created_at DESC
        "#
    )
    .fetch_all(&app_state.pool)
    .await?;

    Ok(Json(blogs))
}
```

Create

```rust
pub async fn create(
    app_state: State<AppState>,
    Json(body): Json<CreateBlogDto>,
) -> Result<(StatusCode, Json<Blog>), BlogError> {
    let blog = sqlx::query_as!(
        Blog,
        r#"
            INSERT INTO blogs (title, content, author_id)
            VALUES ($1, $2, $3)
            RETURNING *
        "#,
        body.title,
        body.content,
        body.author_id
    )
    .fetch_one(&app_state.pool)
    .await?;

    Ok((StatusCode::CREATED, Json(blog)))
}
```

The pattern:

- Need to distinguish error types? → Use map_err
- All errors are the same type (database errors)? → Use simple ?

Blog App

Your turn!